

МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет
имени академика С.П. Королева»
(Самарский университет)

Институт информатики и кибернетики

Лабораторная работа №1

По дисциплине «Параллельное программирование»

Выполнил студент гр. 6312

Бисултанов Умар З.

Самара 2026

Цель работы

Написать алгоритм на языке C++, который проводит перемножение двух квадратным матриц одного размера, обработку файлов и верификацию результатов путем использования готовых операций из библиотеки numru.

Теория

Основная формула: $c_{ij} = \sum_{k=1}^n a_{ik}b_{kj}$,

где С – матрица результат , А – первая матрица , В – вторая матрица.

Количество операций при умножении двух матриц определяется $2N^3$,

где N – размерность матрицы.

Поэтапное описание алгоритма

1. Считывает исходные матрицы.
2. Проверяет совместимость размерностей.
3. Перемножает.
4. Измеряет время выполнения программы.
5. Сохраняет матрицу результат в файл.
6. Выводит размер матрицы, количество операций и время выполнения.
7. Верификация: результат выполняется сторонней программой, чтобы убедиться в верности.

Исходный программный код

```
#include <iostream>
#include <fstream>
#include <vector>
#include <chrono>

using namespace std;

vector<vector<double>> readMatrix(string filename, int &size) {
    ifstream file(filename);
    file >> size;

    vector<vector<double>> matrix(size, vector<double>(size));

    for (int i = 0; i < size; i++)
        for (int j = 0; j < size; j++)
            file >> matrix[i][j];

    return matrix;
}

void writeMatrix(string filename, vector<vector<double>> &matrix) {
    ofstream file(filename);
    int size = matrix.size();

    file << size << endl;

    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++)
            file << matrix[i][j] << " ";
        file << endl;
    }
}

vector<vector<double>> multiplyMatrix(vector<vector<double>> &A, vector<vector<double>> &B) {
    int size = A.size();
    vector<vector<double>> C(size, vector<double>(size, 0));

    for (int i = 0; i < size; i++)
        for (int j = 0; j < size; j++)
            for (int k = 0; k < size; k++)
```

```
        C[i][j] += A[i][k] * B[k][j];

    return C;
}

int main() {

    int size1, size2;

    auto A = readMatrix("A.txt", size1);
    auto B = readMatrix("B.txt", size2);

    if (size1 != size2) {
        cout << "Not same size\n";
        return 1;
    }

    auto start = chrono::high_resolution_clock::now();

    auto C = multiplyMatrix(A, B);

    auto end = chrono::high_resolution_clock::now();
    chrono::duration<double> elapsed = end - start;

    writeMatrix("result.txt", C);

    long operations = 2 * size1 * size1 * size1;

    cout << "Matrix size: " << size1 << "x" << size1 << endl;
    cout << "Operations: " << operations << endl;
    cout << "Execution time: " << elapsed.count() << " seconds" << endl;

    return 0;
}
```

Формат входных и выходных данных.

На примере первой матрицы – это .txt файл вида

3

2 1 3

0 4 5

7 2 1

Алгоритм верификации результирующих данных

```
import numpy as np

def read_matrix(filename):
    with open(filename) as f:
        n = int(f.readline())
        data = []
        for _ in range(n):
            data.append(list(map(float, f.readline().split())))
    return np.array(data)

A = read_matrix("A.txt")
B = read_matrix("B.txt")
C_cpp = read_matrix("result.txt")

C_np = A @ B

if np.allclose(C_cpp, C_np):
    print("Verification PASSED")
else:
    print("Verification FAILED")
    print("NumPy result:")
    print(C_np)
```

Эксперименты

Размер матрицы	Кол-во операций	Время выполнения в секундах
200	1.6×10^7	0.04
400	1.28×10^8	0.27
800	1.02×10^9	2.3
1200	3.5×10^9	7.5
1600	8.2×10^9	17
2000	1.6×10^{10}	38.7

Корреляция



Рисунок 1 - График: размер расположен по оси x, а время – по y

Вывод

Был написан алгоритм выполнения умножения двух квадратных матриц на языке C++. Проведен эксперимент, заключающийся в нахождении зависимости времени выполнения умножения от размера матриц. А также для достоверности работы алгоритма были использованы сторонние библиотеки, чтобы сравнить результат алгоритма с результатом полученным посредством применения numpy.